

Target Tracking V1.0



Official Website



Version	Publishing date	Description
V1.0	20231013	Publish initially

Catalog

1. PID Principle	3
1.1 PID Algorithm Description	3
1.2 PID Algorithm Working Logic	3
1.2.1 Proportional	4
1.2.2 Integral	5
1.2.3 Derivative	7
2. Color Tracking	9
2.1 Program Logic	9
2.2 Operation Steps	10
2.3 Program Outcome	12
2.4 Launch File Analysis	12
2.5 Python Source Code Analysis	13
3. Tag Tracking	21
3.1 Program Logic	21
3.2 Operation Steps	22
3.3 Program Outcome	23
3.4 Launch File Analysis	24
3.5 Python Source Code Analysis	25
4. KCF Target Tracking	31
4.1 KCF Introduction	31
4.2 Program Logic	31
4.3 Operation Steps	32
4.4 Program Outcome	33
4.5 Launch File Analysis	34
4.6 Python Source Code Analysis	35

1. PID Principle

1.1 PID Algorithm Description

PID algorithm is the most widely used autonomous controller which controls a process according to the ratio of errors, Proportional (P), Integral (I), and Derivative (D). With simple logic, it is easy to realize and can be widely applicable. And its control parameters are separate and it is easy to select the parameter.

$$u(t) = K_p \left(e(t) + \frac{1}{T_i} \int_0^t e(t) dt + T_d \frac{de(t)}{dt} \right)$$

Simplify:

$$u(t) = K_p e(t) + K_i \int_0^t e(t) dt + K_d \frac{de(t)}{dt}$$

And:

$u(t)$ ——the output of the PID controller;

K_p ——ratio factor;

K_i ——integral time constant; $K_i = \frac{K_p}{T_i}$

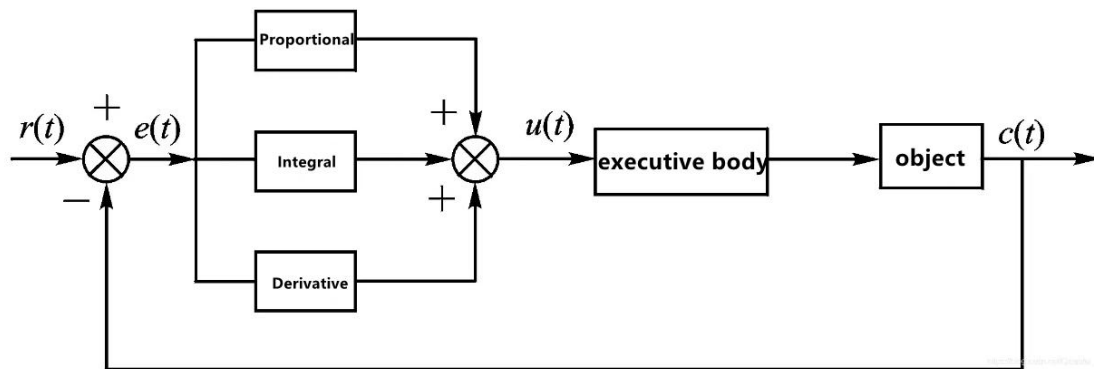
K_d ——derivative time constant; $K_d = K_p \cdot T_d$

$e(t)$ ——the difference between the given value and the measured value (error)

1.2 PID Algorithm Working Logic

In a closed-loop control system, the output result of the object controlled will be returned to affect the output of the controller so as to generate one or multiple loop. The closed-loop system has positive and negative feedback. If the

feedback signal is opposite to the given signal, it is negative feedback. If they are the same, it is positive feedback.



1.2.1 Proportional

Error is the difference between the given value (the set threshold $r(t)$) and the measured output ($c(t)$). After the proportional is introduced, the output of the system is proportional to the error, which can reduce the error quickly and stabilize the system. When the system tends to stabilize, steady-state error will still exist and cannot be eliminated. The control effect of proportional depends on the value of K_p . The smaller the proportional coefficient K_p , the weaker the control and the slower the system response. On the contrary, the larger the proportional coefficient K_p , the stronger the control, and the faster the system response. However, too large K_p will cause the system to generate large overshoot and oscillation. The advantage of proportional control lies in its fast response, but the disadvantage is that there is a steady-state error.

Take a bucket for example. There is a hole at the bottom of the bucket, but you need to maintain the water in the bucket at 1m. And the current water level is 0.2m, and the water will leak 0.1m whenever you add the water. Here, proportional control can be adopted to control the water volume. The current water volume is 0.2m and the target is 1m. Set $K_p=0.4$. Then $u=0.4 \cdot e$, and $e = \text{last water volume} - \text{current water volume}$

The first error: $e(1)=1-0.2=0.8\text{m}$. Therefore, the volume of the added water is 0.32m ($K_p \cdot 0.8 = 0.4 \cdot 0.8 = 0.32\text{m}$), and the current water volume in the bucket

is 0.42m ($0.2+0.32-0.1=0.42\text{m}$)

The second error: $e(2) = 1-(0.2+0.32)=0.48\text{m}$. Therefore, the volume of the added water is 0.192m ($K_p \cdot 0.48=0.192\text{m}$), and the current water volume in the bucket is 0.552m

($0.52+0.192=0.552$)

NO.	Current difference	Output water volumn	Final output water volumn		KP
1			0.2		0.4
2	0.8	0.32	0.42		
3	0.58	0.232	0.552		
4	0.448	0.1792	0.6312		
5	0.3688	0.14752	0.67872		
6	0.32128	0.128512	0.707232		
7	0.292768	0.1171072	0.7243392		
8	0.2756608	0.11026432	0.73460352		
9	0.26539648	0.10615859	0.74076211		
10	0.259237888	0.10369516	0.74445727		
11	0.255542733	0.10221709	0.74667436		
12	0.25332564	0.10133026	0.74800462		
13	0.251995384	0.10079815	0.74880277		
14	0.25119723	0.10047889	0.74928166		
15	0.250718338	0.10028734	0.749569		
16	0.250431003	0.1001724	0.7497414		
17	0.250258602	0.10010344	0.74984484		
18	0.250155161	0.10006206	0.7499069		

Lastly, the water level stabilizes at 0.75m, and the error is 0.25m. When you add 0.1m water, the volume of the water added is equivalent to the volume of the water leaking. And this kind of error is called steady-state error.

Steady-state error is defined as the difference between the output under the system tends to stabilize and the target output.

1.2.2 Integral

Integral aims at eliminating the steady-state error. The integral of the error over time is continuously accumulated to keep the output changing so as to reduce the error. If the integral time is sufficient, the steady-state error can be

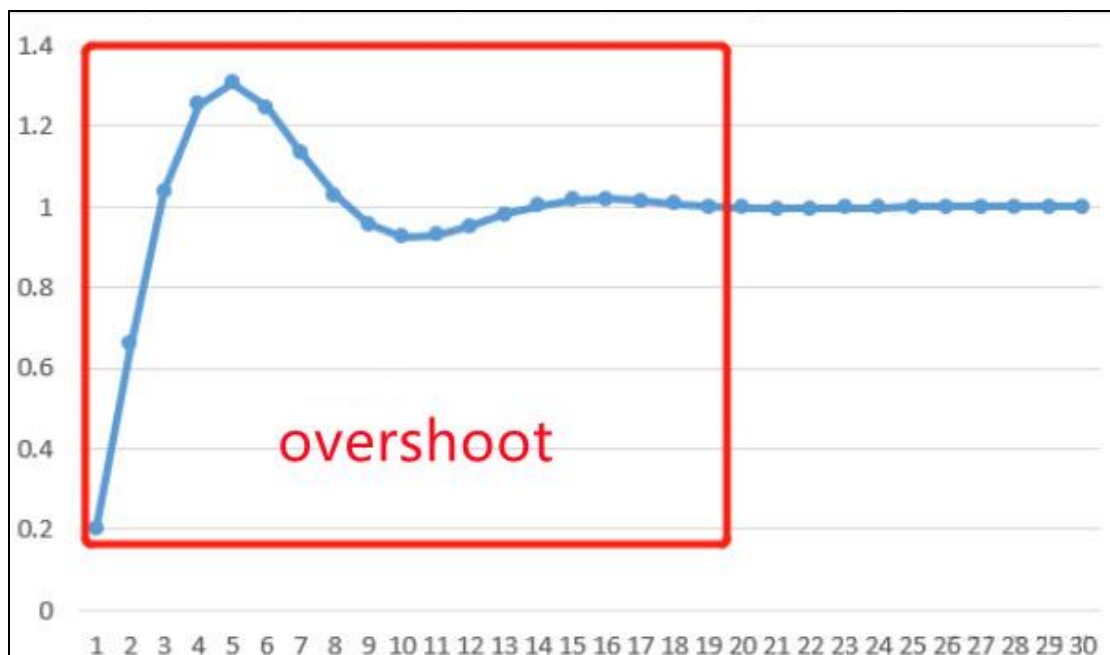
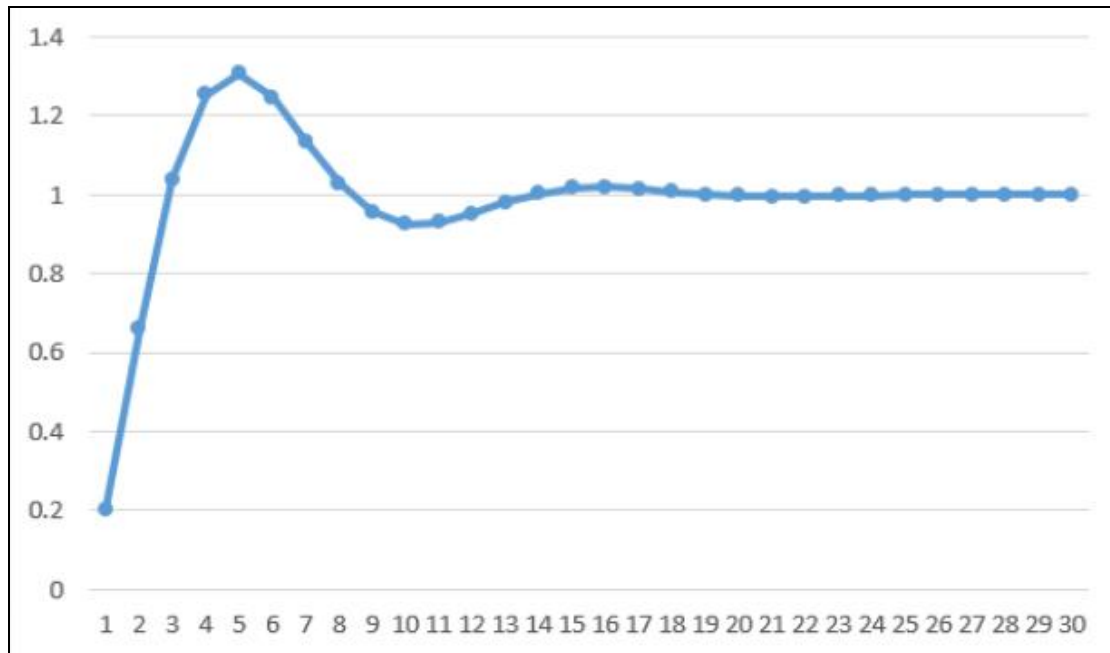
completely eliminated. The smaller T_i is, the stronger the function of integral is. But too strong integral function will make the system overshoot and even oscillate. The advantage of the integral is that the steady-state error can be completely eliminated, but the disadvantage is that the response speed is reduced and the overshoot will be too large.

In the example given in “**2.1 Proportional**”, the steady-state error still exists after proportional control is adopted. To eliminate it completely, you can add integral control, and set the integral constant K_i as 0.3 ($K_i=0.3$).

The first error: $e(1)=1-0.2=0.8m$. Therefore, the volume of the added water is: $K_p \cdot 0.8 = 0.4 \cdot 0.8 = 0.32m$, $K_i \cdot e(1) = 0.3 \cdot 0.8 = 0.24$, and the final added water is: $0.32m + 0.256 = 0.56$. And the current water volume in the bucket is : $0.2 + 0.276 - 0.1 = 0.66$

The second error: $e(2)=1-0.66=0.34m$. Therefore, the volume of the added water is: $K_p \cdot 0.34 = 0.4 \cdot 0.34 = 0.136m$, $K_i \cdot (e(2)+e(1)) = 0.3 \cdot 0.342 = 0.342$, and the final added water is: $0.136m + 0.342 = 0.478$. And the current water volume in the bucket is : $0.66 + 0.478 - 0.1 = 1.038$

NO.	Current difference	KP output value	KI output value	PI output value	final water volumn	KP	KI
1					0.2		
2	0.8	0.32	0.24	0.56	0.66	0.4	0.3
3	0.34	0.136	0.342	0.478	1.038		
4	-0.038	-0.0152	0.3306	0.3154	1.2534		
5	-0.2534	-0.10136	0.25458	0.15322	1.30662		
6	-0.30662	-0.122648	0.162594	0.039946	1.246566		
7	-0.246566	-0.0986264	0.0886242	-0.0100022	1.1365638		
8	-0.1365638	-0.0546255	0.04765506	-0.0069705	1.02959334		
9	-0.02959334	-0.0118373	0.03877706	0.02693972	0.95653306		
10	0.043466938	0.01738678	0.05181714	0.06920391	0.92573698		
11	0.074263023	0.02970521	0.07409605	0.10380126	0.92953823		
12	0.070461768	0.02818471	0.09523458	0.12341928	0.95295752		
13	0.047042484	0.01881699	0.10934732	0.12816432	0.98112183		
14	0.018878168	0.00755127	0.11501077	0.12256204	1.00368387		
15	-0.003683871	-0.0014735	0.11390561	0.11243206	1.01611593		
16	-0.016115934	-0.0064464	0.10907083	0.10262446	1.01874039		
17	-0.018740391	-0.0074962	0.10344871	0.09595256	1.01469295		
18	-0.014692948	-0.0058772	0.09904083	0.09316365	1.0078566		



After the Integral control is adopted, the steady-state error is completely eliminated, but overshoot occurs.

1.2.3 Derivative

The slope error can be obtained by differentiating the error, and the slope reflects the change (rate of change) of the error signal, so the error can be adjusted in advance in derivative part. And it takes effect at the moment when

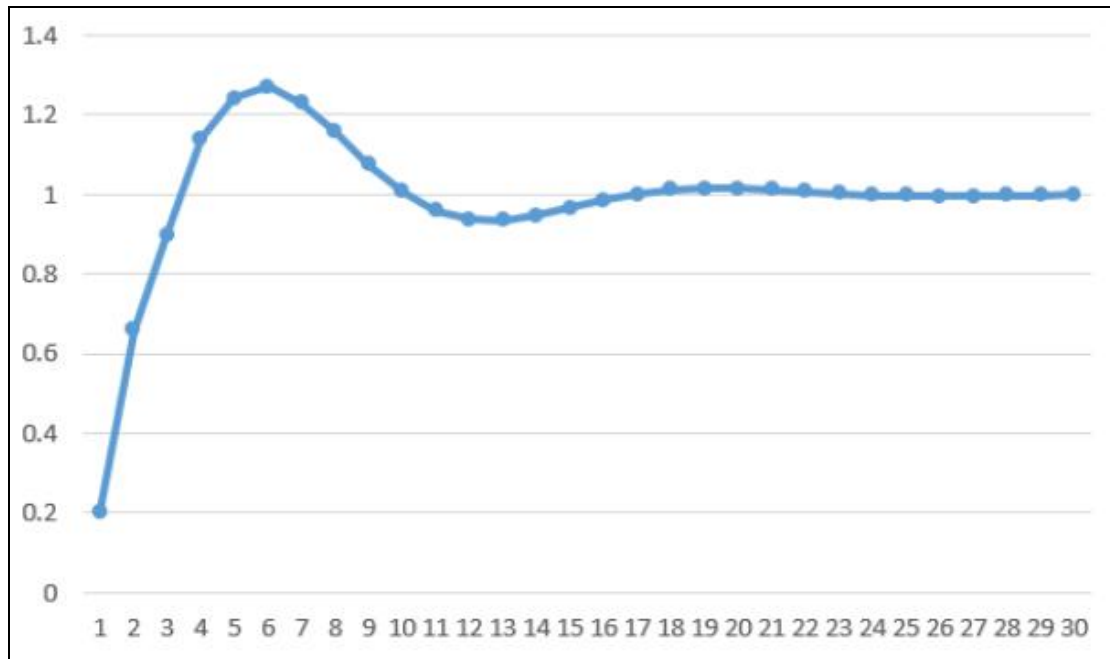
the error appears or changes, so as to avoid excessive overshoot and reduce the dynamic error and response time. The larger the derivative coefficient T_d , the stronger the ability to suppress the error variation. The advantage of derivative is that it suppress overshoot and speed up the response. The disadvantage is that the anti-interference ability is poor. When the interference signal is strong, it is not recommended to add derivative.

In the above example, the integral effectively eliminates the steady-state error, but overshoot occurs. In order to reduce the overshoot, a derivative control is added. Set K_d as 0.3 ($K_d=0.3$).

The first error: $e(1)=1-0.2=0.8m$. Therefore, the volume of the added water is: $K_p \cdot 0.8 = 0.4 \cdot 0.8 = 0.32m$, $K_I \cdot e(1) = 0.3 \cdot 0.8 = 0.24$. And $K_D = 0$ (because the current water difference is 0.8). The final added water is: $0.32m + 0.256 = 0.56$. And the current water volume in the bucket is : $0.2 + 0.276 - 0.1 = 0.66$

The second error: $e(2)=1-0.66=0.34m$. Therefore, the volume of the added water is: $K_p \cdot 0.34 = 0.4 \cdot 0.34 = 0.136m$, $K_I \cdot (e(2)+e(1)) = 0.3 \cdot 0.342 = 0.342$. And $K_D \cdot (e(2) - e(1)) = 0.3 \cdot (0.342 - 0.8) = -0.138$. The final added water is: $0.136m + 0.342 - 0.138 = 0.34$. And the current water volume in the bucket is : $0.66 + 0.478 - 0.1 = 0.9$

次数	与目前的误差	KP输出值	KI输出值	KD输出	PI输出值	最终的水量	KP	KI	KD
1						0.2		0.4	0.3
2	0.8	0.32	0.24	0	0.56	0.66			0.3
3	0.34	0.136	0.342	-0.138	0.34	0.9			
4	0.1	0.04	0.372	-0.072	0.34	1.14			
5	-0.14	-0.056	0.33	-0.072	0.202	1.242			
6	-0.242	-0.0968	0.2574	-0.0306	0.13	1.272			
7	-0.272	-0.1088	0.1758	-0.009	0.058	1.23			
8	-0.23	-0.092	0.1068	0.0126	0.0274	1.1574			
9	-0.1574	-0.06296	0.05958	0.02178	0.0184	1.0758			
10	-0.0758	-0.03032	0.03684	0.02448	0.031	1.0068			
11	-0.0068	-0.00272	0.0348	0.0207	0.05278	0.95958			
12	0.04042	0.016168	0.046926	0.014166	0.07726	0.93684			
13	0.06316	0.025264	0.065874	0.006822	0.09796	0.9348			
14	0.0652	0.02608	0.085434	0.000612	0.112126	0.946926			
15	0.053074	0.0212296	0.1013562	-0.0036378	0.118948	0.965874			
16	0.034126	0.0136504	0.111594	-0.0056844	0.11956	0.985434			
17	0.014566	0.0058264	0.1159638	-0.005868	0.1159222	1.0013562			
18	-0.0013562	-0.0005425	0.11555694	-0.0047767	0.1102378	1.011594			



Compared with proportional control, the overshoot is weakened.

2. Color Tracking

When the camera identifies the target color block (default is red), the robotic arm moves in sync with the color block's movement.

2.1 Program Logic

Firstly, subscribe to the topic messages published by the color recognition node to acquire information about the recognized color.

Next, upon matching the target color, retrieve the central position of the target image.

Finally, employ inverse kinematics to calculate the angles necessary to align the center of the screen with the center of the target image. Publish the corresponding topic message, control servo rotation, and ensure the robotic arm follows the movement of the target.

The source code of the program is stored in

/home/hiwonder/jetarm/src/jetarm_6dof/jetarm_6dof_functions/scripts/color_tracking.py

```

47         center_x = center_x * 4 / w
48     if abs(center_x - 0.5) > 0.02: # 相差范围小于一定值就不用再动了
49         self.pid_yaw.SetPoint = 0.5 # 我们的目标是要让色块在画面的中心，就是整个画面的像素宽度的
50         1/2 位置
51         self.pid_yaw.update(center_x)
52         self.yaw = min(max(self.yaw + self.pid_yaw.output, 0), 1000)
53     else:
54         self.pid_yaw.clear() # 如果已经到达中心了就复位一下 pid 控制器
55
56     center_y = center_y * 4 / h
57     if abs(center_y - 0.5) > 0.02:
58         self.pid_pitch.SetPoint = 0.5
59         self.pid_pitch.update(center_y)
60         self.pitch = min(max(self.pitch + self.pid_pitch.output, 100), 740)
61     else:
62         self.pid_pitch.clear()
63     # rospy.loginfo("x:{:.2f}\ty:{:.2f}".format(self.x, self.y))
64     return (result_image, (self.pitch, self.yaw))
65 else:
66     return (result_image, None)
67

```

2.2 Operation Steps

- 1) Start the robot arm, and access the robot system desktop using NoMachine according to the tutorials saved in '1. Getting Ready /1. JetArm Introductory Manual/11. Remote Desktop System Installation and Connection'.



- 2) Double-click  to open the command-line terminal.
- 3) Execute the command '**sudo systemctl stop jetarm_bringup.service**' and hit Enter to input the password '**hiwonder**' to terminate the auto-start service.

```

> sudo systemctl stop jetarm_bringup.service
[sudo] password for hiwonder:

```

- 4) Enter the command "**roslaunch jetarm_6dof_functions color_tracking.launch**" to launch the game program (here, it defaults to tracking red; if you want to change the color, you can use "**roslaunch jetarm_6dof_functions color_tracking.launch target_color:="blue"**").

```
> roslaunch jetarm_6dof_functions color_tracking.launch target_color:="blue"
```

Adjust the color setting; if tracking a different color, simply change the color name inside target_color.

```
> roslaunch jetarm_6dof_functions color_tracking.launch
```

- 5) Once the game is initiated, you will see the following message printed in the terminal, and at this point, the camera feed will automatically pop up on the screen.

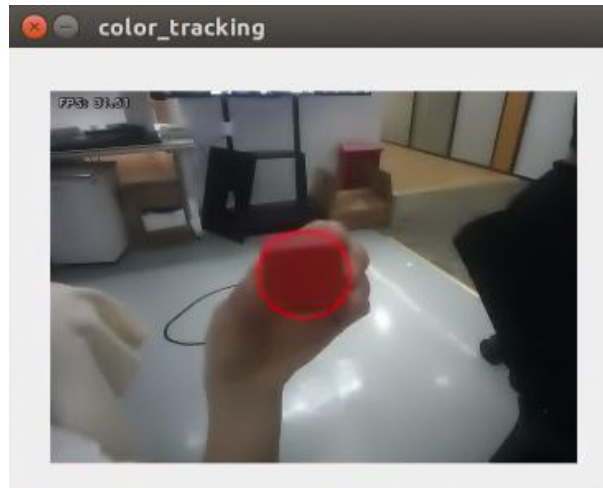
```
[INFO] [1692797345.804477]: kinematics init finish
[DEBUG] [1692797345.817794]: node[/kinematics, http://localhost:39529/] entering spin(), pid[31275]
[INFO] [1692797348.018642]: 正在设置将要追踪的目标认识为 red
[INFO] [1692797348.036558]: 订阅原图像节点 red
[DEBUG] [1692797348.051727]: node[/color_tracking, http://localhost:44235/] entering spin(), pid[31284]
[DEBUG] [1692797348.099756]: connecting to localhost 51991
[ INFO] [1692797348.119910516]: UVCCameraDriver image subscribed
[ INFO] [1692797348.132865203]: set uvc mode 640x480@30 format mjpeg
qt.qpa.xcb: QXcbConnection: XCB error: 148 (Unknown), sequence: 186, resource id : 0, major code: 140 (Unknown), minor code: 20
```

- 6) If you need to terminate the program, please use short-cut '**Ctrl+C**'. If the program fails to stop, please retry.
- 7) After running the previous program, you need to restart the app service by executing this command '**sudo systemctl start jetarm_bringup.service**'. After you input the command, hit Enter and type the password '**hiwonder**'. Then wait for the robot arm to return to the initial pose and the buzzer on the robot to emit a beep sound.

```
> sudo systemctl start jetarm_bringup.service
[sudo] password for hiwonder:
```

2.3 Program Outcome

After starting the game, place a red block in front of the robotic arm's camera. The transmitted image will highlight the recognized target color, and the robotic arm will continuously follow the movement of the target block.



2.4 Launch File Analysis

Launch file is saved in

/home/hiwonder/jetarm/src/jetarm_6dof/jetarm_6dof_functions/launch/color_tracking.launch

```

1 <launch>
2   <arg name="target_color" default="red" />
3   <arg name="source_image_topic" default="/rgbd_cam/color/image_raw" />
4   <include file="$(find jetarm_bringup)/launch/base.launch"/>
5
6   <node name="color_tracking" pkg="jetarm_6dof_functions" type="color_tracking.py" output="screen"
7     respawn="true">
8     <param name="source_image_topic" value="$(arg source_image_topic)" />
9     <param name="target_color" value="$(arg target_color)" />
10  </node>
11 </launch>

```

The **"target_color"** specifies the color to be tracked as "red."

```

2   <arg name="target_color" default="red" />

```

The **"source_image_topic"** defines the image topic name as **"/rgbd_cam/color/image_raw"**.

```

3   <arg name="source_image_topic" default="/rgbd_cam/color/image_raw" />

```

Launch the **"base.launch"** file to initiate the basic control node for the robotic

arm.

```
4 <include file="$(find jetarm_bringup)/launch/base.launch"/>
```

Initiate the 'color_tracking' source code file using the node.

```
6 <node name="color_tracking" pkg="jetarm_6dof_functions" type="color_tracking.py"
  output="screen" respawn="true">
7 <param name="source_image_topic" value="$(arg source_image_topic)" />
8 <param name="target_color" value="$(arg target_color)" />
```

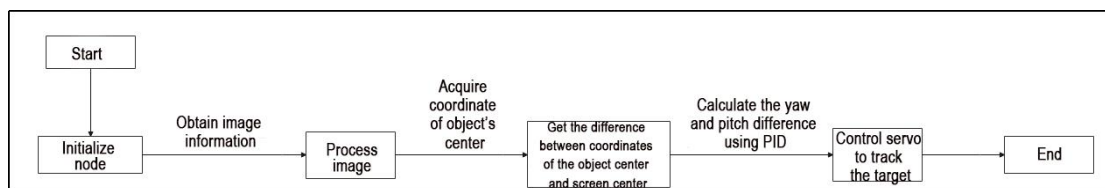
2.5 Python Source Code Analysis

The Python source code is saved in:

/home/hiwonder/jetarm/src/jetarm_6dof/jetarm_6dof_functions/scripts/color_tracking.py

```
47 center_x = center_x * 4 / w
48
49 if abs(center_x - 0.5) > 0.02: # 相差范围小于一定值就不用再动了
50     self.pid_yaw.SetPoint = 0.5 # 我们的目标是要让色块在画面的中心，就是整个画面的像素宽度的
    1/2 位置
51     self.pid_yaw.update(center_x)
52     self.yaw = min(max(self.yaw + self.pid_yaw.output, 0), 1000)
53 else:
54     self.pid_yaw.clear() # 如果已经到达中心了就复位一下 pid 控制器
55
56 center_y = center_y * 4 / h
57 if abs(center_y - 0.5) > 0.02:
58     self.pid_pitch.SetPoint = 0.5
59     self.pid_pitch.update(center_y)
60     self.pitch = min(max(self.pitch + self.pid_pitch.output, 100), 740)
61 else:
62     self.pid_pitch.clear()
63 # rospy.loginfo("x: {:.2f}\ty: {:.2f}".format(self.x, self.y))
64 return (result_image, (self.pitch, self.yaw))
65 else:
66     return (result_image, None)
67
```

The program flow chart is as below:



From the above diagram, the program's logical flow primarily involves processing the acquired image and transmit the live camera feed.

◆ Import Feature Pack

Import the necessary modules using the import statements.


```
4 import os
5 import cv2
6 import rospy
7 import numpy as np
8 from sensor_msgs.msg import Image as RosImage
9 from hiwonder_interfaces.msg import MultiRawIdPosDur
10 from vision_utils import fps, colors, get_area_max_contour
11 from jetarm_sdk import pid, bus_servo_control
```

os provides various interfaces for Python programs to interact with the operating system.

cv2 is used for OpenCV image processing.

rospy is employed for communication in the ROS environment.

numpy is utilized for array operations.

Referenced message types include:

- Image from sensor_msgs.msg.
- MultiRawIdPosDur for servos from hiwonder_interfaces.msg.
- Functions from vision_utils include:
 - fps (frames per second for image transmission),
 - colors,
 - get_area_max_contour (maximum contour).
- In jetarm_sdk, pid and bus_servo_control are utilized for servo control

◆ Launch ObjectTrackingNode Class (Tracking)

Initiate the **ObjectTrackingNode** class and name the tracking node as

"color_tracking."

```
105 if __name__ == "__main__":
106     node = ObjectTrackingNode("color_tracking", log_level=rospy.DEBUG)
```

◆ Initialization Function of ObjectTrackingNode Class


```

69 class ObjectTrackingNode:
70     def __init__(self, node_name, log_level=rospy.INFO):
71         rospy.init_node(node_name, anonymous=True, log_level=log_level)
72
73         self.tracker = None
74         self.fps = fps.FPS()
75         self.color_ranges = rospy.get_param('/config/lab', {})
76         self.servos_pub = rospy.Publisher('/controllers/multi_id_pos_dur', MultiRawIdPosDur,
77             queue_size=1)
78         rospy.sleep(3)
79         bus_servo_control.set_servos(self.servos_pub, 1000, ((1, 500), (2, 700), (3, 85), (4, 350),
80             (5, 500), (10, 200)))
81         rospy.sleep(2)
82         self.target_color = rospy.get_param('~target_color', 'red')
83         rospy.loginfo("正在设置将要追踪的目标识别为" + self.target_color)
84         self.tracker = ColorTracker(self.target_color)
85         self.source_image_topic = rospy.get_param('~source_image_topic', '/camera/image_raw')
86         rospy.loginfo("订阅原图像节点 " + self.target_color)

```

`rospy.init_node()` initializes the ROS node and related variables.

Parameters:

`self.tracker`: Tracking target, initially set to "None."

`self.fps`: Frame rate counter.

`self.color_ranges`: Obtains threshold ranges for various colors from the parameter server.

`self.servos_pub`: Servo node.

`bus_servo_control.set_servos`: Servo control, sets the initial pose of the robotic arm.

`self.target_color`: Tracking color, initially set to "red"; users can modify it here.

`self.tracker`: Initializes color tracking, invoking the `ColorTracker` class.

`self.source_image_topic`: Topic for obtaining camera image information.

`rospy.loginfo`: Subscribes to the original image node.

`self.image_sub`: Publishes image detection, receiving the transmitted feed.

1) Initialize color tracking, and invoke `ColorTracker` class.

```

82 self.tracker = ColorTracker(self.target_color)

```

2) Receive the live camera feed.

```

85 self.image_sub = rospy.Subscriber(self.source_image_topic, RosImage, self.image_callback,
86     queue_size=1)

```

The image detection is published, creating an image subscriber with

`rospy.Subscriber`, where:

The first parameter, **self.source_image_topic**, denotes the topic name for obtaining image information.

The second parameter, **RosImage**, specifies the message type.

The third parameter signifies the invocation of the **image_callback()** callback function.

◆ Initiate ColorTracker Class

```
15 class ColorTracker:
16     def __init__(self, target_color):
17         self.target_color = target_color
18         self.pid_yaw = pid.PID(55.5, 0, 1.2)
19         self.pid_pitch = pid.PID(45.5, 0, 1.2)
20         self.yaw = 500
21         self.pitch = 350
```

The parameter **self.target_color** represents the tracking color, initially set to "red" (to change the default tracking color, modifications can be made here).

```
80 self.target_color = rospy.get_param('~target_color', 'red')
```

The parameter **self.pid_yaw** represents the PID controller (pid.PID(55.5, 0, 1.2)) for controlling the yaw angle of the robotic arm (ID1 servo). In this PID controller, 55.5 corresponds to the parameter setting for P. If the tracking performance of the robotic arm is not satisfactory, it can be fine-tuned downward to a range between 45 and 55.

Similarly, the parameter **self.pid_pitch** is the PID controller for controlling the pitch angle of the robotic arm (ID4 servo), and the adjustment method is the same as mentioned above.

Additionally, **self.yaw** represents the initial position of the yaw angle (ID1 servo), controlling the left-right rotation of the robotic arm. Similarly, **self.pitch** represents the initial position of the pitch angle (ID4 servo), controlling the up-down rotation of the robotic gripper.

◆ Proc Function (Color Tracking)

```
23 def proc (self, source_image, result_image, color_ranges):
24     h, w = source_image.shape[:2]
25     color = color_ranges[self.target_color]
26
27     img = cv2.resize(source_image, (160, 120))
28     img_blur = cv2.GaussianBlur(img, (3, 3), 3) # 高斯模糊
29     img_lab = cv2.cvtColor(img_blur, cv2.COLOR_RGB2LAB) # 转换到 LAB 空间
30     mask = cv2.inRange(img_lab, tuple(color['min']), tuple(color['max'])) # 二值化
```

- 1) Obtain the width and height of the image.

```
24 h, w = source_image.shape[:2]
```

- 2) Acquire the target color range.

```
25 color = color_ranges[self.target_color]
```

- 3) Utilize the `resize()` function from the `cv2` library to scale down the image, reducing computational load.

```
27 img = cv2.resize(source_image, (160, 120))
```

The first parameter, "**source_image**," inside the function parentheses is the input image, and the second parameter, "**(160, 120)**," represents the length and width of the image after scaling.

- 4) Apply Gaussian blur using the `GaussianBlur()` function from the `cv2` library to filter the image and remove noise.

```
28 img_blur = cv2.GaussianBlur(img, (3, 3), 3) # 高斯模糊
```

The first parameter, "`img`," is the input image that has already undergone scaling.

The second parameter, "`(3, 3)`," represents the size of the Gaussian convolution kernel, where both the height and width of the kernel must be positive odd numbers.

The third parameter, "`3`," denotes the standard deviation of the Gaussian kernel in the horizontal direction.

- 5) Convert the image into LAB color space.

```
29 img_lab = cv2.cvtColor(img_blur, cv2.COLOR_RGB2LAB) # 转换到 LAB 空间
```

- 6) Binary processing is performed using the `inRange()` function from the `cv2`

library to threshold the image.

```
30 mask = cv2.inRange(img_lab, tuple(color['min']), tuple(color['max']))
```

The first parameter, "**img_lab**," is the input image.

The second parameter, "**tuple(color['min'])**," represents the minimum value of the color threshold.

The third parameter, "**tuple(color['max'])**," represents the maximum value of the color threshold.

When the RGB value of a pixel falls within the color threshold range, that pixel is assigned a value of "1"; otherwise, it is assigned a value of "0."

7) Erosion and dilation processing is applied to the image, smoothing the contour edges within the image and facilitating the subsequent search for target contours.

```
33 eroded = cv2.erode(mask, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))
34 dilated = cv2.dilate(eroded, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))
```

The `erode()` function is employed for erosion operations on the image. Taking the code "`eroded = cv2.erode(mask, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))`" as an example, the parameters inside the parentheses are explained as follows:

The first parameter, "**mask**," is the input image.

The second parameter, "**cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))**," determines the nature of the operation with a structuring element or kernel. In this case, the first parameter inside the parentheses indicates the kernel shape, and the second parameter indicates the kernel size.

Similarly, the **dilate()** function is used for dilation operations on the image. The meaning of the parameters inside the parentheses for this function is the same as for the **erode()** function.

- 8) To obtain the contour with the maximum area, the `findContours()` function from the `cv2` library is called. This function is used to search for the largest contour corresponding to the color recognized as the target within the image.

```
37 contours = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[-2]
38 max_contour_area = get_area_max_contour(contours, 10)
```

The first parameter, "**dilated**," is the input image.

The second parameter, "**cv2.RETR_EXTERNAL**," represents the contour retrieval mode.

The third parameter, "**cv2.CHAIN_APPROX_NONE**," signifies the contour approximation method.

- 9) Circle the tracked target.

```
45 circle_color = colors.rgb[self.target_color] if self.target_color in colors.rgb else (
46     0x55, 0x55, 0x55)
47 cv2.circle(result_image, (int(center_x * 4), int(center_y * 4)), int(radius * 4),
48     circle_color, 2)
49 center_x = center_x * 4 / w
```

- 10) Update PID data through PID algorithm, and update X-axis and Y-axis coordinates.

```
49 if abs(center_x - 0.5) > 0.02: # 相差范围小于一定值就不用再动了
50     self.pid_yaw.SetPoint = 0.5 # 我们的目标是要让色块在画面的中心,
51     # 就是整个画面的像素宽度的 1/2 位置
52     self.pid_yaw.update(center_x)
53     self.yaw = min(max(self.yaw + self.pid_yaw.output, 0), 1000)
54 else:
55     self.pid_yaw.clear() # 如果已经到达中心了就复位一下 pid 控制器
56     center_y = center_y * 4 / h
57 if abs(center_y - 0.5) > 0.02:
58     self.pid_pitch.SetPoint = 0.5
59     self.pid_pitch.update(center_y)
60     self.pitch = min(max(self.pitch + self.pid_pitch.output, 100), 740)
61 else:
62     self.pid_pitch.clear()
```

When the x-axis coordinate of the center of the target color (`center_x`) minus the x-axis coordinate of the screen center (0.5) is greater than 0.02, the object is not at the center of the screen. When it is less than 0.02, it is considered to be at the center of the screen.

When the object is not at the center of the screen, the yaw angle (ID4) of the robotic arm is adjusted using the PID_yaw controller. After the adjustment, the value of self.yaw is recalculated using the function `min(max(self.yaw + self.pid_yaw.output, 0), 1000)` to obtain the new yaw angle. This involves taking the maximum between the target value and 0, and then taking the minimum between the result and 1000. This ensures that the yaw angle (ID4 servo) is within the specified range.

When the object is at the center of the screen, the PID controller is reset directly using the `clear()` function.

The same principles apply to pitch angle adjustments.

- 1) Return the image info, the offset of pitch angle, and yaw angle.

```
64 | ... | ... | return (result_image, (self.pitch, self.yaw))
```

◆ image_callback Callback Function

Process the image information, and display the live camera feed.

```
87 | def image_callback(self, ros_image: RosImage):
88 |     # rospy.logdebug('Received an image! ')
89 |     # 将ros格式图像转换为opencv格式
90 |     rgb_image = np.ndarray(shape=(ros_image.height, ros_image.width, 3), dtype=np.uint8, buffer=
ros_image.data)
91 |     result_image = np.copy(rgb_image)
92 |
93 |     if self.tracker is not None:
94 |         color_ranges = rospy.get_param('/config/lab', self.color_ranges)
95 |         result_image, p_y = self.tracker.proc(rgb_image, result_image, color_ranges)
96 |         if p_y is not None:
97 |             bus_servo_control.set_servos(self.servos_pub, 30, ((1, p_y[1]), (4, p_y[0])))
98 |         # 计算帧率及发布结果图像
99 |         self.fps.update()
100 |         result_image = self.fps.show_fps(result_image)
101 |         cv2.imshow("color_tracking", cv2.resize(cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR), (320, 240
))) # 显示小尺寸会更快更节省资源, 想要大把resize去掉
102 |         cv2.waitKey(1)
```

- 2) Read the image information using `np.ndarray()`.

```
87 | def image_callback(self, ros_image: RosImage):
88 |     # rospy.logdebug('Received an image! ')
89 |     # 将ros格式图像转换为opencv格式
90 |     rgb_image = np.ndarray(shape=(ros_image.height, ros_image.width, 3), dtype=np.uint8, buffer=
ros_image.data)
```

- 3) Pass the image to 'result_image'.

```
91 | result_image = np.copy(rgb_image)
```

- 4) If `self.tracker` is not 'None,' then retrieve the threshold ranges for each

color and the processed image information from the parameter server.

```
93 if self.tracker is not None:
94     color_ranges = rospy.get_param('/config/lab', self.color_ranges)
95     result_image, p_y = self.tracker.proc(rgb_image, result_image, color_ranges)
```

5) If **p_y** is not '**None**', proceed to initiate color tracking.

```
97 bus_servo_control.set_servos(self.servos_pub, 30, ((1, p_y[1]), (4, p_y[0])))
```

The first parameter, **self.servos_pub**, is responsible for publishing information related to servo control.

The second parameter 30 represents the delay time: 30ms.

The third parameter ((1, p_y[1]), (4, p_y[0])) represents the offset positions for Servo ID1 and Servo ID4.

6) Calculate the frame rate and publish the result image, and display the image using **cv2.imshow**.

```
99 self.fps.update()
100 result_image = self.fps.show_fps(result_image)
101 cv2.imshow("color tracking", cv2.resize(cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR), (320,
102     240))) # 显示小尺寸会更快更节省资源, 想要大把resize去掉
102 cv2.waitKey(1)
```

3. Tag Tracking

When the camera detects the target tag (default: ID2), the robotic arm will follow the movement of the tag.

3.1 Program Logic

Firstly, subscribe to the topic messages published by the tag tracking node to obtain information about the recognized tag.

Then, upon identifying the target tag, retrieve the central position of the tag image.

Finally, use inverse kinematics to calculate the angles required to align the center of the screen with the center of the target image. Publish the corresponding topic messages to control servo rotation, enabling the robotic

arm to follow the target movement.

The source code for this program can be found at:

/home/hiwonder/jetarm/src/jetarm_6dof/jetarm_6dof_functions/scripts/tag_tracking.py

```

13 class TagTrackingNode:
14     def __init__(self):
15         rospy.init_node('tag_tracking_node')
16
17         self.pid_yaw = pid.PID(20.5, 0, 7.5)
18         self.pid_pitch = pid.PID(17.0, 0, 7.5)
19         self.yaw = 500
20         self.pitch = 350
21
22         self.tracker = None
23         self.enable_select = False
24
25         self.at_detector = Detector(searchpath=['apriltags'],
26                                     families='tag36h11',
27                                     nthreads=8,
28                                     quad_decimate=2.0,
29                                     quad_sigma=0.0,
30                                     refine_edges=1,
31                                     decode_sharpening=0.25,
32                                     debug=0)
33
34         self.fps = fps.FPS() # 帧率统计器
35         self.servos_pub = rospy.Publisher('/controllers/multi_id_pos_dur', MultiRawIdPosDur, queue_size=1)
36         rospy.sleep(3)
37         bus_servo_control.set_servos(self.servos_pub, 1000, ((1, 500), (2, 700), (3, 85), (4, 350), (5,
38         500), (10, 200)))
39         rospy.sleep(2)

```

3.2 Operation Steps

- 1) Start the robot arm, and access the robot system desktop using NoMachine according to the tutorials saved in '1. Getting Ready /1. JetArm Introductory Manual/11. Remote Desktop System Installation and Connection'.



- 2) Double-click  to open the command-line terminal.
- 3) Execute the command '**sudo systemctl stop jetarm_bringup.service**' and hit Enter to input the password '**hiwonder**' to terminate the auto-start service.

```

> sudo systemctl stop jetarm_bringup.service
[sudo] password for hiwonder:

```

- 4) Execute the command '**roslaunch jetarm_6dof_functions tag_tracking.launch**' to run the game program.

```
> roslaunch jetarm_6dof_functions tag_tracking.launch
```

- 5) After initiating the game, the terminal will display the following message, and the live camera feed will start automatically.

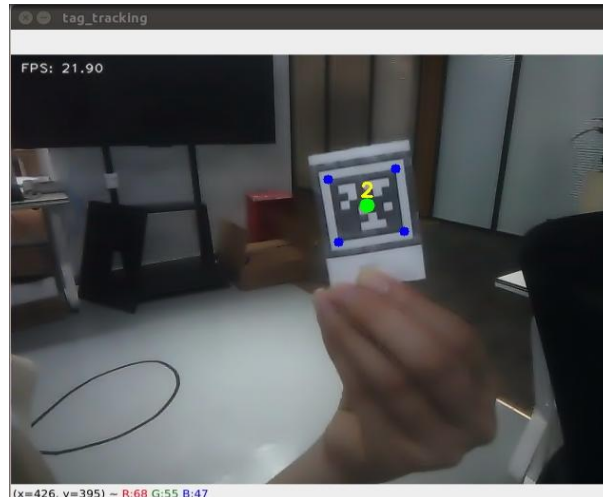
```
[DEBUG] [1692788344.349381]: ... service URL is rosrpc://localhost:39025
[DEBUG] [1692788344.355529]: [/kinematics/set_joint_value_target]: new Service i
nstance
[INFO] [1692788344.379777]: kinematics init finish
[DEBUG] [1692788344.389489]: node[/kinematics, http://localhost:43765/] entering
spin(), pid[16172]
[INFO] [1692788345.595378]: 订阅原图像节点 /rgbd_cam/color/image_raw
[ INFO] [1692788345.643624523]: UVCCameraDriver image subscribed
[ INFO] [1692788345.655222439]: set uvc mode 640x480@30 format mjpeg
qt.qpa.xcb: QXcbConnection: XCB error: 148 (Unknown), sequence: 186, resource id
: 0, major code: 140 (Unknown), minor code: 20
```

- 6) If you need to terminate the game, press '**Ctrl+C**'. If the program cannot be stopped, please retry.
- 7) After running the previous program, you need to restart the app service by executing this command '**sudo systemctl start jetarm_bringup.service**'. After you input the command, hit Enter and type the password '**hiwonder**'. Then wait for the robot arm to return to the initial pose and the buzzer on the robot to emit a beep sound.

```
> sudo systemctl start jetarm_bringup.service
[sudo] password for hiwonder:
~
```

3.3 Program Outcome

After starting the game, place the ID2 tag in front of the robotic arm's camera. The screen will display the recognized tag ID, and the robotic arm will continuously follow the movement of the ID2 tag.



3.4 Launch File Analysis

The Launch file is saved in

/home/hiwonder/jetarm/src/jetarm_6dof/jetarm_6dof_functions/launch/tag_tracking.launch

```
1 <launch>
2   <arg name="source_image_topic" default="/rgbd_cam/color/image_raw" />
3   <include file="$(find jetarm_bringup)/launch/base.launch"/>
4
5   <node name="kcf_tracking" pkg="jetarm_6dof_functions" type="tag_tracking.py" output="screen"
6     respawn="true">
7     <param name="source_image_topic" value="$(arg source_image_topic)" />
8   </node>
9 </launch>
```

“**source_image_topic**”has defined the image topic name as “**/rgbd_cam/color/image_raw**”.

```
2   <arg name="source_image_topic" default="/rgbd_cam/color/image_raw" />
```

Initiate ‘**base.launch**’ file to invoke the robot arm basic control node.

```
3   <include file="$(find jetarm_bringup)/launch/base.launch"/>
```

Launch the ‘**tag_tracking**’ source code file through the node.

```
5   <node name="kcf_tracking" pkg="jetarm_6dof_functions" type="tag_tracking.py" output="screen"
6     respawn="true">
7     <param name="source_image_topic" value="$(arg source_image_topic)" />
8   </node>
```

3.5 Python Source Code Analysis

Python source code file is saved in

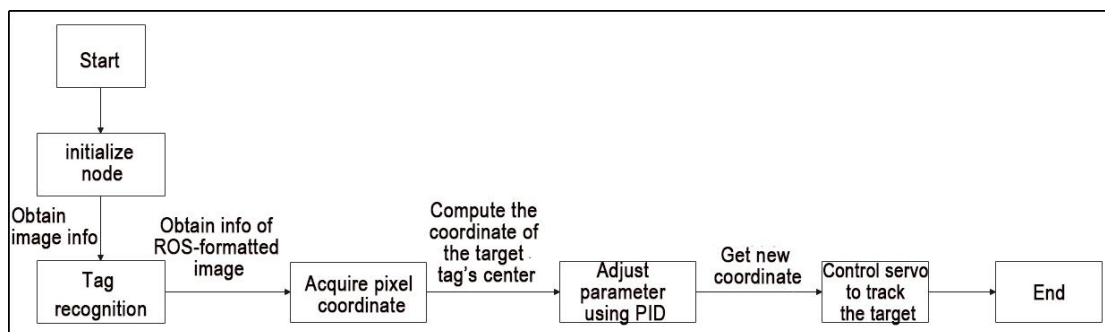
/home/hiwonder/jetarm/src/jetarm_6dof/jetarm_6dof_functions/scripts/tag_tracking.py

```

13 class TagTrackingNode:
14     def __init__(self):
15         rospy.init_node('tag_tracking_node')
16
17         self.pid_yaw = pid.PID(20.5, 0, 7.5)
18         self.pid_pitch = pid.PID(17.0, 0, 7.5)
19         self.yaw = 500
20         self.pitch = 350
21
22         self.tracker = None
23         self.enable_select = False
24
25         self.at_detector = Detector(searchpath=['apriltags'],
26                                   families='tag36h11',
27                                   nthreads=8,
28                                   quad_decimate=2.0,
29                                   quad_sigma=0.0,
30                                   refine_edges=1,
31                                   decode_sharpening=0.25,
32                                   debug=0)
33
34         self.fps = fps.FPS() # 帧率统计器
35         self.servos_pub = rospy.Publisher('/controllers/multi_id_pos_dur', MultiRawIdPosDur, queue_size=1)
36         rospy.sleep(3)
37         bus_servo_control.set_servos(self.servos_pub, 1000, ((1, 500), (2, 700), (3, 85), (4, 350), (5,
38         500), (10, 200)))
39         rospy.sleep(2)

```

The program's logical flowchart is illustrated in the diagram obtained by organizing information from the source files.



The main logic flow of the program involves processing the captured images and displaying the live feed.

◆ Import Feature Pack

Import the required module using import statement.


```

2  import os
3  import cv2
4  import rospy
5  import numpy as np
6  from sensor_msgs.msg import Image
7  from vision_utils import fps, draw_tags
8  from hiwonder_interfaces.msg import MultiRawIdPosDur
9  from dt_apriltags import Detector
10 from jetarm_sdk import bus_servo_control, pid

```

os provides various interfaces for Python programs to interact with the operating system;

cv2 is used for OpenCV image processing;

rospy is used for ROS communication;

numpy is used for array operations;

sensor_msgs.msg is referenced for the Image message type;

vision_utils is used for fps (frames per second of image transmission) and draw_tags;

hiwonder_interfaces.msg is referenced for the MultiRawIdPosDur message type (servos);

Detector is used from dt_apriltags;

bus_servo_control and pid (servo control) are utilized from jetarm_sdk.

◆ Initiate TagTrackingNode Class (Tracking)

Initialize the TagTrackingNode class and name the tracking node as 'tag_tracking_node'.

```

14 def __init__(self):
15     rospy.init_node('tag_tracking_node')

```

◆ Initialization Function of TagTrackingNode 类的初始化函数

1) Initialize PID parameter.

```

17 self.pid_yaw = pid.PID(20.5, 0, 7.5)
18 self.pid_pitch = pid.PID(17.0, 0, 7.5)
19 self.yaw = 500
20 self.pitch = 350

```


The parameter **self.pid_yaw** corresponds to the PID controller (pid.PID(20.5, 0, 7.5)) used to control the yaw angle of the robotic arm (servo motor ID1).

Here, the value 20.5 represents the proportional (P) parameter setting. If the tracking performance of the robotic arm is not satisfactory, it is recommended to fine-tune this value to a range between 15 and 20.5.

Similarly, the parameter **self.pid_pitch** is associated with the PID controller (adjustment method as above) responsible for controlling the pitch angle of the robotic arm (servo motor ID4).

The parameters **self.yaw** and **self.pitch** represent the initial positions of the yaw and pitch angles, respectively, for the servo motors ID1 and ID4.

2) Initialize tag tracking.

```
22 self.tracker = None
23 self.enable_select = False
```

3) Define Apriltag.

```
25 self.at_detector = Detector(searchpath=['apriltags'],
26                             families='tag36h11',
27                             nthreads=8,
28                             quad_decimate=2.0,
29                             quad_sigma=0.0,
30                             refine_edges=1,
31                             decode_sharpening=0.25,
32                             debug=0)
```

families: Desired tag families; examples include tag25h9, tag36h11, etc.

nthreads: Number of threads.

quad_decimate: Allows detection of quadrilaterals on lower-resolution images to improve speed.

quad_sigma: Used for Gaussian blur to segment images.

refine_edges: Refines edges for alignment.

decode_sharpening: Sharpens tag decoding.

debug: Debug parameter; does not save debug information.

4) Initialize the frame rate counter and robot arm's original position.

```

34         self.fps = fps.FPS() # 帧率统计器
35         self.servos_pub = rospy.Publisher('/controllers/multi_id_pos_dur', MultiRawIdPosDur,
36         queue_size=1)
37         rospy.sleep(3)
38         bus_servo_control.set_servos(self.servos_pub, 1000, ((1, 500), (2, 700), (3, 85), (4, 350),
39         (5, 500), (10, 200)))
40         rospy.sleep(2)

```

The parameter **self.fps** represents the frame rate counter.

The parameter **self.servos_pub** is for the servo node.

The method **bus_servo_control.set_servos** is used for servo control, establishing the initial pose of the robotic arm.

5) Subscribe to the camera image topic.

```

41         source_image_topic = rospy.get_param('~source_image_topic', 'camera/image_rect_color')
42         rospy.loginfo("订阅原图像节点 " + source_image_topic)

```

The variable **source_image_topic** captures the topic for obtaining camera image information.

The **rospy.loginfo** function subscribes to the original image node.

6) Initiate target tracking by publishing the detection image.

```

43         self.image_sub = rospy.Subscriber(source_image_topic, Image, self.image_callback, queue_size
44         =2)

```

The variable **self.image_sub** receives the transmitted image for detection publishing, creating an image subscriber using '**rospy.Subscriber**' with the following parameters:

The first parameter '**source_image_topic**' represents the topic name for obtaining image information.

The second parameter '**Image**' indicates the message type.

The third parameter represents the invocation of the '**image_callback()**' callback function.

The fourth parameter '**queue_size**' denotes the size of the image queue.

◆ image_callback Callback Function

Process the image information to display live camera feed.

```

46 def image_callback(self, ros_image):
47     #rospy.logdebug('Received an image! ')
48     # 将画面转为 opencv 格式
49     rgb_image = np.ndarray(shape=(ros_image.height, ros_image.width, 3), dtype=np.uint8, buffer=
ros_image.data)
50     result_image = np.copy(rgb_image)
51     factor = 4
52     #rgb_image = cv2.resize(rgb_image, (int(rgb_image[1] / factor), int(rgb_image[0] / factor)))
53     tags = self.at_detector.detect(cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY), False, None, 0.025)
54     tags = sorted(tags, key=lambda tag: tag.tag_id) # 貌似出来就是升序排列的不需要手动进行排列
55     draw_tags(result_image, tags, corners_color=(0, 0, 255), center_color=(0, 255, 0))
56     if len(tags) > 0 and tags[0].tag_id == 2:
57         center_x, center_y = tags[0].center
58         center_x = center_x / rgb_image.shape[1]
59         if abs(center_x - 0.5) > 0.02: # 相差范围小于一定值就不用再动了
60             self.pid_yaw.SetPoint = 0.5 # 我们的目标是让色块在画面的中心, 就是整个画面的像素宽度的
1/2 位置
61             self.pid_yaw.update(center_x)
62             self.yaw = min(max(self.yaw + self.pid_yaw.output, 0), 1000)
63         else:
64             self.pid_yaw.clear() # 如果已经到达中心了就复位一下 pid 控制器
65

```

7) Read image information using np.ndarray().

```

46 def image_callback(self, ros_image):
47     #rospy.logdebug('Received an image! ')
48     # 将画面转为 opencv 格式
49     rgb_image = np.ndarray(shape=(ros_image.height, ros_image.width, 3), dtype=np.uint8, buffer=
ros_image.data)

```

8) Pass the image info to 'result_image'.

```

50 result_image = np.copy(rgb_image)

```

9) The tags variable invokes tag detection function **detect ()** to detect the recognized tag ID.

```

53 tags = self.at_detector.detect(cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY), False, None,
0.025)
54 tags = sorted(tags, key=lambda tag: tag.tag_id) # 默认升序排列的不需要手动进行排列
55 draw_tags(result_image, tags, corners_color=(0, 0, 255), center_color=(0, 255, 0))

```

10) Check if the tag ID is greater than 0 and equal to 2 (users can modify the tracked tag ID at this point).

```

56 if len(tags) > 0 and tags[0].tag_id == 2:

```

11) Retrieve the central coordinates of the X and Y axes for the target tag code.

```

57 center_x, center_y = tags[0].center
58 center_x = center_x / rgb_image.shape[1]

```

12) Check whether the coordinates of the tag's center point and the screen's center point are within the range of 0 to 0.02. If they are, the robotic arm does not need to move. If they are not, the robotic arm moves to the center point of the tag.

```

56         if len(tags) > 0 and tags[0].tag_id == 2:
57             center_x, center_y = tags[0].center
58             center_x = center_x / rgb_image.shape[1]
59             if abs(center_x - 0.5) > 0.02: # 相差范围小于一定值就不用再动了
60                 self.pid_yaw.SetPoint = 0.5 # 我们的目标是要让色块在画面的中心，就是整个画面的像素宽度的
                    1/2 位置
61                 self.pid_yaw.update(center_x)
62                 self.yaw = min(max(self.yaw + self.pid_yaw.output, 0), 1000)
63             else:
64                 self.pid_yaw.clear() # 如果已经到达中心了就复位一下 pid 控制器
65
66             center_y = center_y / rgb_image.shape[0]
67             if abs(center_y - 0.5) > 0.02:
68                 self.pid_pitch.SetPoint = 0.5
69                 self.pid_pitch.update(center_y)
70                 self.pitch = min(max(self.pitch + self.pid_pitch.output, 100), 740)
71             else:
72                 self.pid_pitch.clear()
73             bus_servo_control.set_servos(self.servos_pub, 50, ((1, self.yaw), (4, self.pitch)))

```

Based on the obtained center X-axis coordinate (**center_x**) of the tag code and the X-axis coordinate of the screen center (**0.5**), if the difference is greater than **0.02**, adjust the yaw angle (**ID4**) of the robotic arm using **pid_yaw**. After adjustment, re-output the value of **self.yaw** using the function **min(max(self.yaw + self.pid_yaw.output, 0), 1000)**, where the yaw angle (**servo motor ID4**) is determined by taking the maximum between the target value and 0, and then the minimum between the target value and 1000. If the difference is less than 0.02, reset the PID controller using the **clear()** function.

The pitch angle follows a similar procedure.

13) Control the robot arm to track using **self.yaw** and **self.pitch** functions.

```

71         else:
72             self.pid_pitch.clear()
73             bus_servo_control.set_servos(self.servos_pub, 50, ((1, self.yaw), (4, self.pitch)))

```

The first parameter, **self.servos_pub**, is responsible for publishing information related to servo control.

The second parameter, **50**, represents a delay of 50 milliseconds.

The third parameter, **((1, self.yaw), (4, self.pitch))**, denotes the offset positions for servo ID1 (yaw) and servo ID4 (pitch).

14) Display the live camera feed on the terminal.

```

75         self.fps.update()
76         self.fps.show_fps(result_image)
77         result_image = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)
78         cv2.imshow("tag_tracking", result_image)
79         key = cv2.waitKey(1)

```

4. KCF Target Tracking

Utilizing the KCF algorithm, select the target object by drawing a bounding box with the mouse. When the camera detects the highlighted target object, the system will track its movement and follow it as it moves.

4.1 KCF Introduction

KCF (Kernel Correlation Filter), also known as the Kernelized Correlation Filter algorithm, was proposed by Joao F. Henriques, Rui Caseiro, Pedro Martins, and Jorge Batista in 2014.

KCF is a discriminative tracking method that operates by collecting positive and negative samples using a circulant matrix around the target region. It trains a target detector, and subsequently uses this detector to check whether the predicted position in the next frame is the target. Based on the updated detection results, the training set is then updated, iteratively refining the target detector.

4.2 Program Logic

Firstly, subscribe to the topic messages published by the KCF tracking node to obtain information about the recognized KCF-tracked objects.

Next, check if the 'S' key on the keyboard is pressed.

When 'S' is pressed, use the mouse to draw a bounding box around the region of interest (ROI).

Obtain the ROI and calculate the coordinates of its center.

Finally, employ inverse kinematics to determine the angles necessary to align the center of the screen with the center of the target image. Publish the corresponding topic message to control servo rotation, enabling the robotic

arm to follow the moving target.

The source code of the program is saved in

/home/hiwonder/jetarm/src/jetarm_6dof/jetarm_6dof_functions/scripts/kcf_tracking.py

```

83         except Exception as e:
84             rospy.logerr(str(e))
85
86
87         self.fps.update()
88         self.fps.show_fps(result_image)
89         result_image = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)
90         cv2.imshow("image", result_image)
91
92         key = cv2.waitKey(1)
93         if key == ord('s'): # 按下s开始选择追踪目标
94             self.tracker = None
95             self.enable_select = True
96
97
98 if __name__ == '__main__':
99     try:
100         kcf_tracking = KCFTrackingNode()
101         print("在画面窗口按下s开始选择追踪目标")
102         rospy.spin()
103     except Exception as e:
104         rospy.logerr(str(e))

```

4.3 Operation Steps

- 1) Start the robot arm, and access the robot system desktop using NoMachine according to the tutorials saved in '1. Getting Ready /1. JetArm Introductory Manual/11. Remote Desktop System Installation and Connection'.



- 2) Double-click  to open the command-line terminal.
- 3) Execute the command '**sudo systemctl stop jetarm_bringup.service**' and hit Enter to input the password '**hiwonder**' to terminate the auto-start service.

```

> sudo systemctl stop jetarm_bringup.service
[sudo] password for hiwonder:

```

- 4) Run the command '**roslaunch jetarm_6dof_functions kcf_tracking.launch**' to initiate the game program.

```

> roslaunch jetarm_6dof_functions kcf_tracking.launch

```

- 5) After initializing the game, you will observe the following prompt messages in the terminal, and at this point, the live camera feed will automatically appear.

```
[DEBUG] [1692789702.714033]: [/kinematics/set_joint_value_target]: new Service i
nstance
[INFO] [1692789702.724161]: kinematics init finish
[DEBUG] [1692789702.737005]: node[/kinematics, http://localhost:38971/] entering
spin(), pid[31651]
[INFO] [1692789703.835963]: 订阅原图像节点 /rgbd_cam/color/image_raw
在画面窗口按下s开始选择追踪目标
[ INFO] [1692789703.872396400]: UVCCameraDriver image subscribed
[ INFO] [1692789703.877039577]: set uvc mode 640x480@30 format mjpeg
qt.qpa.xcb: QXcbConnection: XCB error: 148 (Unknown), sequence: 186, resource id
: 0, major code: 140 (Unknown), minor code: 20
```

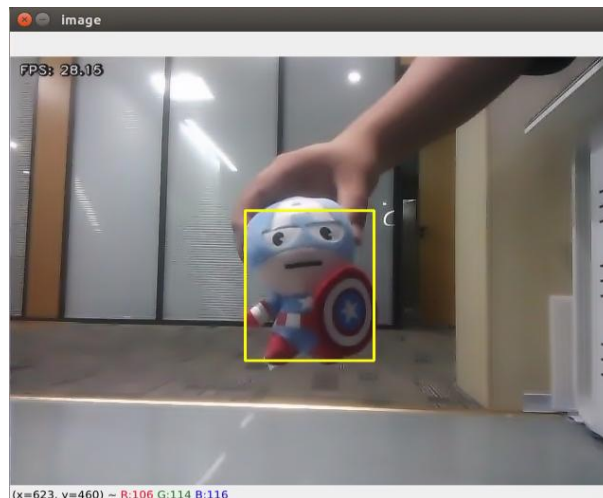
- 6) If you need to terminate the game, press '**Ctrl+C**'. If the program cannot be stopped, please retry.
- 7) After running the previous program, you need to restart the app service by executing this command '**sudo systemctl start jetarm_bringup.service**'. After you input the command, hit Enter and type the password '**hiwonder**'. Then wait for the robot arm to return to the initial pose and the buzzer on the robot to emit a beep sound.

```
> sudo systemctl start jetarm_bringup.service
[sudo] password for hiwonder:
>
```

4.4 Program Outcome

After initiating the game, place the object you want to track in front of the camera. In the live camera feed window, press the 'S' key on the keyboard, then use the left mouse button to draw a bounding box around the object. Finally, press Enter. At this point, the robotic arm will continuously follow the selected object's movement. (Note: It's advisable to keep the object's movement speed moderate. If the object is lost, press 'S' again to re-select and

frame the object.)



4.5 Launch File Analysis

Launch file is saved in

/home/hiwonder/jetarm/src/jetarm_6dof/jetarm_6dof_functions/launch/kcf_tracking.launch

```
1 <launch>
2   <arg name="source_image_topic" default="/rgb_d_cam/color/image_raw" />
3   <include file="$(find jetarm_bringup)/launch/base.launch"/>
4
5   <node name="kcf_tracking" pkg="jetarm_6dof_functions" type="kcf_tracking.py" output="screen"
6     respawn="true">
7     <param name="source_image_topic" value="$(arg source_image_topic)" />
8   </node>
9 </launch>
```

Define the image topic name as **'/rgb_d_cam/color/image_raw'** with the variable **'source_image_topic.'**

```
2 <arg name="source_image_topic" default="/rgb_d_cam/color/image_raw" />
```

Launch the **'base.launch'** file to invoke the basic control node for the robotic arm.

```
3 <include file="$(find jetarm_bringup)/launch/base.launch"/>
```

Initiate the **'kcf_tracking'** source code file through the node.

```
5 <node name="kcf_tracking" pkg="jetarm_6dof_functions" type="kcf_tracking.py" output="screen"
6   respawn="true">
7   <param name="source_image_topic" value="$(arg source_image_topic)" />
8 </node>
```

4.6 Python Source Code Analysis

The Python source code file is saved in

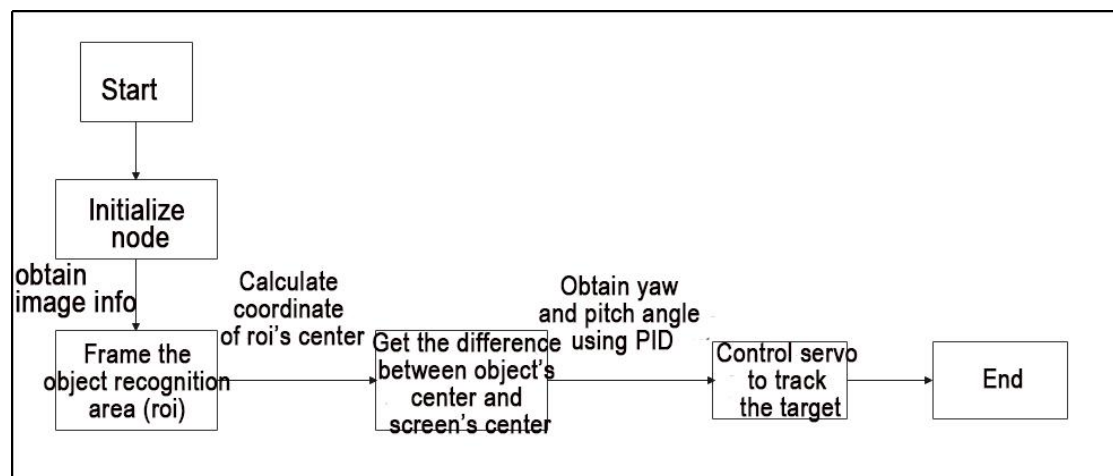
/home/hiwonder/jetarm/src/jetarm_6dof/jetarm_6dof_functions/scripts/kcf_tracking.py

```

83         except Exception as e:
84             rospy.logerr(str(e))
85
86
87         self.fps.update()
88         self.fps.show_fps(result_image)
89         result_image = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)
90         cv2.imshow("image", result_image)
91
92         key = cv2.waitKey(1)
93         if key == ord('s'): # 按下s开始选择追踪目标
94             self.tracker = None
95             self.enable_select = True
96
97
98 if __name__ == '__main__':
99     try:
100         kcf_tracking = KCFTrackingNode()
101         print("在画面窗口按下s开始选择追踪目标")
102         rospy.spin()
103     except Exception as e:
104         rospy.logerr(str(e))

```

The program flowchart is as pictured below:



The main logic flow of the program involves processing the captured images and displaying the live feed.

◆ Import Function Package

Import the required module using import statement.

```
2 import os
3 import cv2
4 import rospy
5 import queue
6 import numpy as np
7 from sensor_msgs.msg import Image
8 from vision_utils import fps, box_center
9 from hiwonder_interfaces.msg import MultiRawIdPosDur
10 from jetarm_sdk import pid, bus_servo_control
```

os provides various interfaces for Python programs to interact with the operating system;

cv2 is used for OpenCV image processing;

rospy is used for ROS communication;

numpy is used for array operations;

sensor_msgs.msg is referenced for the Image message type;

vision_utils is used for fps (frames per second of image transmission) and draw_tags;

hiwonder_interfaces.msg is referenced for the MultiRawIdPosDur message type (servos);

Detector is used from dt_apriltags;

bus_servo_control and pid (servo control) are utilized from jetarm_sdk.

◆ Initiate KCFTTrackingNode Class (Tracking)

```
98 if __name__ == '__main__':
99     try:
100         kcf_tracking = KCFTTrackingNode()
101         print("在画面窗口按下s开始选择追踪目标")
102         rospy.spin()
103     except Exception as e:
104         rospy.logerr(str(e))
```

◆ Initialization Function of KCFTTrackingNode Class

- 1) Initialize the TagTrackingNode class and name the tracking node as 'kcf_node.'


```

13 class KCFTrackingNode:
14     def __init__(self):
15         rospy.init_node('kcf_node')

```

2) Initialize PID parameter.

```

17         self.pid_yaw = pid.PID(15.5, 0, 3.5)
18         self.pid_pitch = pid.PID(15.5, 0, 3.5)
19         self.yaw = 500
20         self.pitch = 350

```

The parameter **self.pid_yaw** represents the PID controller for controlling the yaw angle (servo ID1) of the robotic arm, with the settings **pid.PID(15.5, 0, 3.5)**, where 15.5 indicates the proportional (P) parameter. If the tracking performance of the robotic arm is unsatisfactory, it is recommended to make appropriate downward adjustments to the range between 10 and 15.5.

Similarly, the parameter **self.pid_pitch** is associated with the PID controller for controlling the pitch angle (servo ID4) of the robotic arm, using the same adjustment approach.

The parameters **self.yaw** and **self.pitch** represent the initial positions of the yaw and pitch angles for servo motors ID1 and ID4, respectively.

3) Initialize KCF tracking.

```

22         self.tracker = None
23         self.enable_select = False
24         self.fps = fps.FPS() # 帧率统计器

```

4) Initialize the robot arm's original position.

```

25         self.servos_pub = rospy.Publisher('/controllers/multi_id_pos_dur', MultiRawIdPosDur,
26         queue_size=1)
27         rospy.sleep(3)
28         bus_servo_control.set_servos(self.servos_pub, 1000, ((1, 500), (2, 700), (3, 85), (4, 350),
29         (5, 500), (10, 200)))
30         rospy.sleep(2)

```

The parameter **self.fps** represents the frame rate counter.

The parameter **self.servos_pub** is for the servo node.

The method **bus_servo_control.set_servos** is used for servo control, establishing the initial pose of the robotic arm.

5) Subscribe to the camera image topic.

```
31 self.image_queue = queue.Queue(maxsize=2)
32 source_image_topic = rospy.get_param('~source_image_topic', 'camera/image_rect_color')
33 rospy.loginfo("订阅原图像节点 " + source_image_topic)
```

The variable **source_image_topic** is used to obtain camera image information from the topic.

The **rospy.loginfo** parameter subscribes to the original image node.

6) Publish the detection image to initiate target tracking.

```
34 self.image_sub = rospy.Subscriber(source_image_topic, Image, self.image_callback, queue_size=2)
```

The variable **self.image_sub** receives the live camera feed's detection image publication and creates an image subscriber using '**rospy.Subscriber**,' where:

The first parameter, '**source_image_topic**,' denotes the topic name for obtaining image information.

The second parameter, '**Image**,' signifies the message type.

The third parameter indicates the invocation of the '**image_callback()**' function.

The fourth parameter, '**queue_size**,' restricts the number of messages in the queue.

◆ image_callback Callback Function

Process the image information for target tracking

```
37 def image_callback(self, ros_image):
38     #rospy.logdebug('Received an image! ')
39     # 将画面转为 opencv 格式
40     rgb_image = np.ndarray(shape=(ros_image.height, ros_image.width, 3), dtype=np.uint8, buffer=
41     ros_image.data)
42     result_image = np.copy(rgb_image)
43     factor = 4
44     rgb_image = cv2.resize(rgb_image, (int(ros_image.width / factor), int(ros_image.height / factor)))
45     try:
46         if self.tracker is None:
47             if self.enable_select:
48                 roi = cv2.selectROI("image", cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR), False)
49                 roi = tuple(int(i / factor) for i in roi)
50                 if roi:
51                     param = cv2.TrackerKCF.Params()
52                     param.detect_thresh = 0.2
53                     self.tracker = cv2.TrackerKCF_create(param)
54                     self.tracker.init(rgb_image, roi)
55             else:
56                 status, box = self.tracker.update(rgb_image)
```

7) Read the image information using np.ndarray()

```

37 def image_callback(self, ros_image):
38     #rospy.logdebug('Received an image! ')
39     # 将画面转为 opencv 格式
40     rgb_image = np.ndarray(shape=(ros_image.height, ros_image.width, 3), dtype=np.uint8, buffer=
        ros_image.data)

```

8) Pass the image info to 'result_image'.

```

41 result_image = np.copy(rgb_image)

```

9) Invoke cv2.resize () function to resize the image.

```

42 factor = 4
43 rgb_image = cv2.resize(rgb_image, (int(ros_image.width / factor), int(ros_image.height /
        factor)))

```

10) Select ROI (region of interest) area.

```

45 try:
46     if self.tracker is None:
47         if self.enable_select:
48             roi = cv2.selectROI("image", cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR),
                False)
49             roi = tuple(int(i / factor) for i in roi)

```

Firstly, check if self.tracker is 'None' (initially set to None).

If it is 'None,' then check if self.enable_select is True (becomes True when 'S' is pressed).

If it is True, use the **cv2.selectROI()** function to allow the user to outline the region of interest for subsequent cropping and processing.

Next, convert the RGB image to BGR; then, use a tuple to obtain the four vertices of the ROI area.

11) When there are parameters in the ROI, construct a KCF tracker and pass the parameters to self.tracker.

```

50 if roi:
51     param = cv2.TrackerKCF.Params()
52     param.detect_thresh = 0.2
53     self.tracker = cv2.TrackerKCF_create(param)
54     self.tracker.init(rgb_image, roi)

```

12) If the self.tracker is not None, launch KCF tracking.

```

55 else:
56     status, box = self.tracker.update(rgb_image)
57     if status:
58         # rospy.loginfo(str(box))
59         p1 = int(box[0] * factor), int(box[1] * factor)
60         p2 = p1[0] + int(box[2] * factor), p1[1] + int(box[3] * factor)
61         cv2.rectangle(result_image, p1, p2, (255, 255, 0), 2)
62         center_x, center_y = (p1[0] + p2[0]) / 2, (p1[1] + p2[1]) / 2
63         # print(center_x, center_y, result_image.shape[:2], rgb_image.shape[:2])
64
65         center_x = center_x / result_image.shape[1]
66         if abs(center_x - 0.5) > 0.02: # 相差范围小于一定值就不用再动了
67             self.pid_yaw.SetPoint = 0.5 # 我们的目标是要让色块在画面的中心,
68             # 就是整个画面的像素宽度的 1/2 位置
69             self.pid_yaw.update(center_x)
70             self.yaw = min(max(self.yaw + self.pid_yaw.output, 0), 1000)
71         else:
72             self.pid_yaw.clear() # 如果已经到达中心了就复位一下 pid 控制器
73
74         center_y = center_y / result_image.shape[0]
75         if abs(center_y - 0.5) > 0.02:
76             self.pid_pitch.SetPoint = 0.5
77             self.pid_pitch.update(center_y)
78             self.pitch = min(max(self.pitch + self.pid_pitch.output, 100), 740)
79         else:
80             self.pid_pitch.clear()
81         bus_servo_control.set_servos(self.servos_pub, 30, ((1, self.yaw), (4, self.pitch)))
82         # rospy.loginfo("x:{:.2f}\ty:{:.2f}".format(self.x, self.y))

```

- Obtain ROI (region of interest) area.

```

56 status, box = self.tracker.update(rgb_image)

```

- Calculate the coordinate of the four corners of ROI, and frame it using the blue box, then calculate the coordinate of the tracked target's center.

```

59 p1 = int(box[0] * factor), int(box[1] * factor)
60 p2 = p1[0] + int(box[2] * factor), p1[1] + int(box[3] * factor)
61 cv2.rectangle(result_image, p1, p2, (255, 255, 0), 2)
62 center_x, center_y = (p1[0] + p2[0]) / 2, (p1[1] + p2[1]) / 2

```

- Check if the deviation range is less than 0.02. If it is, it indicates that the tag block is positioned at the center of the screen, and the robotic arm does not need to move. Update the PID data and, through the PID algorithm, update the coordinates of the X and Y axes.

```

66 if abs(center_x - 0.5) > 0.02: # 相差范围小于一定值就不用再动了
67     self.pid_yaw.SetPoint = 0.5 # 我们的目标是要让色块在画面的中心,
68     # 就是整个画面的像素宽度的 1/2 位置
69     self.pid_yaw.update(center_x)
70     self.yaw = min(max(self.yaw + self.pid_yaw.output, 0), 1000)
71 else:
72     self.pid_yaw.clear() # 如果已经到达中心了就复位一下 pid 控制器
73
74 center_y = center_y / result_image.shape[0]
75 if abs(center_y - 0.5) > 0.02:
76     self.pid_pitch.SetPoint = 0.5
77     self.pid_pitch.update(center_y)
78     self.pitch = min(max(self.pitch + self.pid_pitch.output, 100), 740)

```

Compare the obtained center position coordinates of the ROI region with the coordinates of the screen's center point to obtain the deviation. Update the PID data information accordingly, and finally, use the output result to update the

coordinates of the X and Y axes for the tag code

13) If it is greater than 0.02, the robot arm will start tracking the target.

```
78         else:
79             self.pid_pitch.clear()
80             bus_servo_control.set_servos(self.servos_pub, 30, ((1, self.yaw), (4, self.pitch)))
81             # rospy.loginfo("x:{:.2f}\ty:{:.2f}".format(self.x, self.y))
```

The first parameter, **self.servos_pub**, is responsible for publishing information related to servo control.

The second parameter, **30**, represents a delay of 30 milliseconds.

The third parameter, **((1, self.yaw), (4, self.pitch))**, denotes the offset positions for servo ID1 (yaw) and servo ID4 (pitch).

14) The live camera feed will be displayed on the terminal.

```
87         self.fps.update()
88         self.fps.show_fps(result_image)
89         result_image = cv2.cvtColor(result_image, cv2.COLOR_RGB2BGR)
90         cv2.imshow("image", result_image)
```

15) Check if the 'S' key on the keyboard is pressed. If pressed, set **self.tracker** to None and **self.enable_select** to True (ROI area selection is only allowed when 'S' is pressed in the feedback screen).

```
92         key = cv2.waitKey(1)
93         if key == ord('s'): # 按下s开始选择追踪目标
94             self.tracker = None
95             self.enable_select = True
```